

Java 8 Features

Java 8, released in 2014, was the largest change to the language in its history. The main additions are lambda expressions, functional interfaces, method references, the Stream API, Optional, default methods in interfaces, and a new date and time API.

A good order to learn them in:

```
Lambda → Functional interface → Method reference → Stream  
→ Optional → Default method → Date and Time
```

1. Lambda expression

A lambda is a short way to write a block of code that can be passed to a method. Before Java 8 this needed an anonymous class.

```
import java.util.ArrayList;  
import java.util.Collections;  
import java.util.Comparator;  
import java.util.List;  
  
public class Main {  
    public static void main(String[] args) {  
        List<String> names = new ArrayList<>();  
        names.add("Ravi");  
        names.add("Amit");  
        names.add("Priya");  
  
        // Before Java 8  
        Collections.sort(names, new Comparator<String>() {  
            @Override  
            public int compare(String a, String b) {  
                return a.compareTo(b);  
            }  
        });  
        System.out.println(names);  
  
        // With a lambda  
        names.sort((a, b) -> b.compareTo(a));  
        System.out.println(names);  
    }  
}
```

Output:

```
[Amit, Priya, Ravi]
[Ravi, Priya, Amit]
```

Ten lines became one. The shape is:

```
(parameters) -> { body }
```

The short forms drop what the compiler can work out:

```
(String a, String b) -> { return a.compareTo(b); } // full
(a, b) -> a.compareTo(b) // types and return dropped
name -> name.length() // one parameter, no brackets
() -> System.out.println("Hi") // no parameter
```

2. Functional interface

A functional interface is an interface with exactly one abstract method. A lambda can only be assigned to one, because the lambda becomes that method.

```
@FunctionalInterface
interface Calculator {
    int operate(int a, int b);
}

public class Main {
    public static void main(String[] args) {
        Calculator add = (a, b) -> a + b;
        Calculator multiply = (a, b) -> a * b;

        System.out.println(add.operate(5, 3));
        System.out.println(multiply.operate(5, 3));
    }
}
```

Output:

```
8
15
```

The `@FunctionalInterface` annotation is optional. It makes the compiler check that there is exactly one abstract method, so a second one added later becomes a compile error instead of a puzzle.

Java supplies the interfaces we need most often in `java.util.function`:

Interface	Method	Takes	Returns
Predicate<T>	test	one value	boolean
Function<T, R>	apply	one value	another value
Consumer<T>	accept	one value	nothing
Supplier<T>	get	nothing	one value

```
import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.function.Supplier;

public class Main {
    public static void main(String[] args) {
        Predicate<Integer> isEven = n -> n % 2 == 0;
        Function<String, Integer> length = s -> s.length();
        Consumer<String> printer = s -> System.out.println("Value: " + s);
        Supplier<String> greeting = () -> "Hello";

        System.out.println(isEven.test(4));
        System.out.println(length.apply("Java"));
        printer.accept("test");
        System.out.println(greeting.get());
    }
}
```

Output:

```
true
4
Value: test
Hello
```

3. Method reference

When a lambda does nothing except call one existing method, a method reference says the same thing with less text.

```
names.forEach(name -> System.out.println(name)); // lambda
names.forEach(System.out::println);             // method reference
```

There are four kinds:

Kind	Syntax	Example
Static method	<code>Class::method</code>	<code>Integer::parseInt</code>
Method of one object	<code>object::method</code>	<code>System.out::println</code>
Method of a type	<code>Class::method</code>	<code>String::toUpperCase</code>
Constructor	<code>Class::new</code>	<code>ArrayList::new</code>

```
import java.util.Arrays;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Ravi", "Amit", "Priya");

        names.forEach(System.out::println);
    }
}
```

Output:

```
Ravi
Amit
Priya
```

4. Stream API

A stream is a pipeline that carries the elements of a collection through a series of steps. A pipeline has three parts:

```
Source      →  Intermediate operations  →  Terminal operation
list        filter, map, sorted          collect, forEach, count
```

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Ravi", "Amit", "Priya", "Anil");

        List<String> result = names.stream()
            .filter(name -> name.startsWith("A"))
            .map(String::toUpperCase)
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

Output:

```
[AMIT, ANIL]
```

Two things matter here.

The stream does not change the original list. `names` still holds the four original values.

Intermediate operations are lazy. `filter` and `map` do nothing when they are written. The whole pipeline runs only when the terminal operation asks for a result.

5. Common stream operations

Operation	Kind	What it does
<code>filter</code>	intermediate	Keeps elements that match
<code>map</code>	intermediate	Turns each element into something else
<code>sorted</code>	intermediate	Orders the elements
<code>distinct</code>	intermediate	Removes duplicates
<code>limit</code>	intermediate	Keeps the first n
<code>count</code>	terminal	Counts the elements
<code>sum</code>	terminal	Totals a number stream
<code>reduce</code>	terminal	Combines everything into one value
<code>anyMatch</code>	terminal	Returns true if any element matches

Operation	Kind	What it does
<code>collect</code>	terminal	Gathers the result into a collection

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(5, 3, 9, 3, 7, 1);

        System.out.println(numbers.stream().distinct().sorted().collect(Collectors.toList()));
        ;
        System.out.println(numbers.stream().filter(n -> n > 4).count());
        System.out.println(numbers.stream().mapToInt(Integer::intValue).sum());
        System.out.println(numbers.stream().reduce(0, (a, b) -> a + b));
        System.out.println(numbers.stream().anyMatch(n -> n > 8));
    }
}
```

Output:

```
[1, 3, 5, 7, 9]
3
28
28
true
```

A stream can be used once. Calling a second terminal operation on the same stream throws `IllegalStateException`, which is why each line above starts a new one.

6. Collecting results

`collect` is the terminal operation that builds a result. `Collectors` holds the ready made ones.

```
import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Ravi", "Amit", "Priya", "Anil", "Rahul");

        String joined = names.stream().collect(Collectors.joining(", "));
        System.out.println(joined);

        Map<Character, List<String>> byFirstLetter = names.stream()
            .collect(Collectors.groupingBy(name -> name.charAt(0),
                TreeMap::new,
                Collectors.toList()));

        System.out.println(byFirstLetter);
    }
}
```

Output:

```
Ravi, Amit, Priya, Anil, Rahul
{A=[Amit, Anil], P=[Priya], R=[Ravi, Rahul]}
```

`groupingBy` sorts the names into buckets by whatever the lambda returns:

```

      names
     /  |  \
    A   P   R
   / \  |  / \
Amit Anil Priya Ravi Rahul
```

The `TreeMap::new` argument is what keeps the keys in order. Without it the grouping goes into a `HashMap` and the printed order is not defined.

Other common collectors are `Collectors.toList()`, `Collectors.toSet()` and `Collectors.counting()`.

7. Optional

`Optional` is a container that either holds a value or is empty. A method that returns `Optional` says plainly that the value might be missing, instead of returning null and leaving the caller to guess.

```
import java.util.Optional;

public class Main {
    public static void main(String[] args) {
        Optional<String> present = Optional.of("Ravi");
        Optional<String> missing = Optional.empty();

        System.out.println(present.isPresent());
        System.out.println(present.get());
        System.out.println(missing.orElse("Unknown"));
        System.out.println(present.map(String::toUpperCase).orElse("Unknown"));
        System.out.println(missing.map(String::toUpperCase).orElse("Unknown"));
    }
}
```

Output:

```
true
Ravi
Unknown
RAVI
Unknown
```

The useful part is the last two lines. `map` runs only when a value is there, so a chain of steps needs no null check at each stage.

Method	What it does
<code>Optional.of(v)</code>	Wraps a value, throws if it is null
<code>Optional.ofNullable(v)</code>	Wraps a value that may be null
<code>Optional.empty()</code>	Creates an empty <code>Optional</code>
<code>isPresent()</code>	Returns true if a value is there
<code>orElse(other)</code>	Returns the value, or <code>other</code> if empty
<code>orElseThrow()</code>	Returns the value, or throws if empty
<code>map(f)</code>	Applies <code>f</code> if a value is there
<code>ifPresent(c)</code>	Runs <code>c</code> if a value is there

So:

`Optional` = a value that might not be there, said out loud

8. Default and static methods in interfaces

Before Java 8 an interface could only declare methods with no body. Adding a method to a published interface broke every class that implemented it. A default method has a body, so existing classes keep compiling.

```
interface Vehicle {
    void start();

    default void stop() {
        System.out.println("Vehicle stopped");
    }

    static String category() {
        return "Transport";
    }
}

class Car implements Vehicle {
    @Override
    public void start() {
        System.out.println("Car started");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();

        car.start();
        car.stop();
        System.out.println(Vehicle.category());
    }
}
```

Output:

```
Car started
Vehicle stopped
Transport
```

`Car` never wrote `stop()`, and it still has one.

This is how `Collection` gained `forEach` and `stream` in Java 8 without breaking any existing code.

If two interfaces supply the same default method, Java will not compile until the class says which one it wants:

```
class Report implements Printable, Exportable {
    @Override
    public void render() {
        Printable.super.render();
    }
}
```

9. Date and Time API

The old `java.util.Date` and `Calendar` were mutable, unsafe to share between threads, and full of surprises. Months started at 0, so December was month 11.

Java 8 added `java.time`, where every type is immutable. Any method that looks like it changes a date returns a new one instead.

Type	What it represents
<code>LocalDate</code>	A date, with no time
<code>LocalTime</code>	A time, with no date
<code>LocalDateTime</code>	A date and a time
<code>Period</code>	A gap in years, months and days
<code>Duration</code>	A gap in hours, minutes and seconds
<code>DateTimeFormatter</code>	A pattern for printing or reading a date

```
import java.time.LocalDate;
import java.time.Period;
import java.time.format.DateTimeFormatter;

public class Main {
    public static void main(String[] args) {
        LocalDate joining = LocalDate.of(2020, 6, 15);
        LocalDate today = LocalDate.of(2026, 9, 14);

        System.out.println(joining);
        System.out.println(joining.plusDays(10));
        System.out.println(joining.getMonth());

        Period served = Period.between(joining, today);
        System.out.println(served.getYears() + " years " + served.getMonths() + "
months");

        System.out.println(today.format(DateTimeFormatter.ofPattern("dd-MM-yyyy")));
    }
}
```

Output:

```
2020-06-15
2020-06-25
JUNE
6 years 2 months
14-09-2026
```

`joining` still holds 15 June 2020 after `plusDays(10)` ran, because `plusDays` returned a new date and we only printed it.

The main features together

Feature	What it gives us	Written as
Lambda	Code passed to a method	<code>(a, b) -> a + b</code>
Functional interface	The type a lambda fits into	<code>Predicate</code> , <code>Function</code>
Method reference	A shorter lambda	<code>System.out::println</code>
Stream	A pipeline over a collection	<code>.filter().map().collect()</code>
Optional	A value that might be missing	<code>Optional<Customer></code>
Default method	Growing an interface safely	<code>default void stop()</code>
<code>java.time</code>	Dates that cannot be changed	<code>LocalDate.of(...)</code>

For example, imagine an order processing application. Orders arrive as a list. We total them with a stream, pull out the large ones with `filter` , look up a single order with `Optional` , and hold the order date as a `LocalDate` .

First the class the list holds:

```
import java.time.LocalDate;

class Order {
    private final String customer;
    private final double amount;
    private final LocalDate placedOn;

    Order(String customer, double amount, LocalDate placedOn) {
        this.customer = customer;
        this.amount = amount;
        this.placedOn = placedOn;
    }

    String getCustomer()    { return customer; }
    double getAmount()      { return amount; }
    LocalDate getPlacedOn() { return placedOn; }
}
```

Then the work:

```

import java.time.LocalDate;
import java.util.Arrays;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<Order> orders = Arrays.asList(
            new Order("Ravi", 1200.0, LocalDate.of(2026, 9, 1)),
            new Order("Amit", 450.0, LocalDate.of(2026, 9, 3)),
            new Order("Ravi", 800.0, LocalDate.of(2026, 9, 7)));

        double total = orders.stream()
            .mapToDouble(Order::getAmount)
            .sum();
        System.out.println("Total: " + total);

        List<String> largeOrders = orders.stream()
            .filter(order -> order.getAmount() > 500)
            .map(Order::getCustomer)
            .collect(Collectors.toList());
        System.out.println("Large orders: " + largeOrders);

        Optional<Order> amitOrder = orders.stream()
            .filter(order -> order.getCustomer().equals("Amit"))
            .findFirst();
        System.out.println("Amit paid: " +
            amitOrder.map(Order::getAmount).orElse(0.0));
    }
}

```

Output:

```

Total: 2450.0
Large orders: [Ravi, Ravi]
Amit paid: 450.0

```

Seven lines of pipeline replace three loops, and none of them declare a counter or a temporary list.

The most useful thing at this point is not to memorize the method names. Take a list of objects from any project and rewrite one loop over it as a stream.